School of Computer Science & Engineering
Department of Computer Engineering and Technology

T. Y. B. Tech (CSE)
AY 2026-27 Semester 5

Computer Networks Laboratory (CSE20110)
Assignment 4

Name:	**PRN:**	**Batch:**
Ishaan Vijaykumar Dhore	1262242310	G2

Title: Write a program to simulate the Distance Vector Routing Algorithm.

Objectives

- To understand the working principle of the Distance Vector Routing Algorithm.
- To compute the shortest path and minimum routing cost between nodes using distance vectors.

Theory

Concept of Distance Vector Routing and Bellman–Ford Method

Distance Vector Routing (DVR):

It is a routing method where each router maintains a routing table. The table contains the distance (cost) and next hop to reach every destination. Routers share their routing information with their neighboring routers. The route with the minimum cost is selected.

Bellman-Ford Method:

It is used to find the shortest path in Distance Vector Routing.

`Formula: Dx(y) = minv { C(x,v) + Dv(y) }`

Where:

- $D_x(y)$ = cost from router x to destination y
- C(x,v) = cost from x to neighboring router v
- $D_v(y)$ = cost from v to destination y

Distance Vector Routing Algorithm and Routing Table

Algorithm:

- Each router knows the cost to its directly connected neighbors. It creates a routing table.
- Routers exchange their tables with neighbors.
- Each router calculates new minimum-cost paths using Bellman-Ford.
- The table is updated if a shorter path is found.
- This continues until the network converges (no more changes).

The routing table contains: Destination, Cost, Next Hop — for example:

Destination	**Cost**	**Next Hop**
A	0	—

B	2	B
C	5	B
D	7	B

Distance Vector Routing Example (Routing Table Update)

Consider the following network:

```
      2          3
-----------------  ----------
     A            B
                  C

Here, the numbers represent the cost (distance) between
routers.
Cost from A to B = 2
```

Step 1: Find the path from A to C

There are two possible paths:

- Path 1: Direct path A → C, Cost = 8
- Path 2: Through router B, A → B → C, Cost = Cost(A,B) + Cost(B,C) = 2 + 3 = 5

Step 2: Compare the costs

- Direct path A → C = 8
- Path through B = 5

Since 5 < 8, the router chooses the path through B. Therefore, the shortest path is A → B → C; minimum cost = 5.

Step 3: Routing Table of Router A

Destination	Cost	Next Hop
A	0	—
B	2	B
C	5	B

Distance Vector Routing selects the path with the minimum cost. In this example, instead of directly sending data from A to C with a cost of 8, router A sends the data through B, giving a total cost of 5.

Pseudocode

```
BEGIN
  INPUT number of routers N
  DECLARE cost[N][N]
  DECLARE distance[N][N]
  DECLARE nextHop[N][N]

  PRINT "Enter the network cost matrix"
  PRINT "Use INF if two routers are not directly connected"

  FOR i <- 0 TO N - 1
     FOR j <- 0 TO N - 1
         INPUT cost[i][j]
```

```
        distance[i][j] <- cost[i][j]
        IF i = j THEN
            distance[i][j] <- 0
            nextHop[i][j] <- i
        ELSE IF cost[i][j] != INF THEN
            nextHop[i][j] <- j
        ELSE
            nextHop[i][j] <- -1
        END IF
    END FOR
END FOR

PRINT "Initial Cost Matrix"
DISPLAY cost

PRINT "Initial Routing Tables"
DISPLAY distance and nextHop for every router

REPEAT
    updated <- FALSE
    FOR source <- 0 TO N - 1
        FOR destination <- 0 TO N - 1
            FOR neighbour <- 0 TO N - 1
                IF cost[source][neighbour] != INF AND
                   distance[neighbour][destination] != INF THEN
                   newCost <- cost[source][neighbour]
                             + distance[neighbour][destination]
                    IF newCost < distance[source][destination] THEN
                       distance[source][destination] <- newCost
                       nextHop[source][destination] <- neighbour
                       updated <- TRUE
                   END IF
                END IF
            END FOR
        END FOR
    END FOR
UNTIL updated = FALSE

PRINT "Final Routing Tables"
FOR source <- 0 TO N - 1
    PRINT "Routing Table for Router", source
    FOR destination <- 0 TO N - 1
        PRINT destination,
              distance[source][destination],
              nextHop[source][destination]
    END FOR
END FOR

INPUT selectedSource
INPUT selectedDestination

IF distance[selectedSource][selectedDestination] = INF THEN
    PRINT "No path exists between the selected routers"
ELSE
    PRINT "Minimum Cost:",
          distance[selectedSource][selectedDestination]
    PRINT "Shortest Path:"
    current <- selectedSource
    PRINT current
    WHILE current != selectedDestination
        current <- nextHop[current][selectedDestination]
        IF current = -1 THEN
```

```
            PRINT "Path cannot be determined"
            STOP
        END IF
        PRINT "->", current
    END WHILE
  END IF

END
```

Program Code

Language used: Python

The program below implements the Distance Vector Routing algorithm using the Bellman-Ford method. It accepts a cost matrix for N routers, iteratively computes the minimum-cost routing table for every router (repeating until no further updates occur), and then lets the user query the shortest path and minimum cost between any two chosen routers.

```
# Distance Vector Routing Algorithm (Bellman-Ford based)

INF = float('inf')


def print_matrix(title, matrix, names):
    print("\n" + title)
    for i in range(len(names)):
        row = []
        for j in range(len(names)):
            row.append("INF" if matrix[i][j] == INF else str(matrix[i][j]))
        print(names[i], ":", row)


def main():
    n = int(input("Enter the number of routers: "))
    names = []
    for i in range(n):
        names.append(input(f"Enter name of router {i + 1}: "))

    # Step 1: Input the cost matrix
    print("\nEnter the network cost matrix (use 999 for INF / not connected)")
    cost = [[0] * n for _ in range(n)]
    for i in  range(n):
        for j in range(n):
            value = int(input(f"Cost {names[i]} -> {names[j]}: "))
            cost[i][j] = INF if value == 999 else value

    # Step 2: Initialize distance and next-hop tables
    distance = [row[:] for row in cost]
    next_hop = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                distance[i][j] = 0
                next_hop[i][j] = i
            elif cost[i][j] != INF:
                next_hop[i][j] = j
            else:
                next_hop[i][j] = -1

    print_matrix("Initial Cost Matrix", cost, names)
```

```
    # Step 3: Bellman-Ford - keep updating until no change happens
    updated = True
    while updated:
        updated = False
        for  src in range(n):
            for dst in range(n):
                for nb in range(n):
                    if cost[src][nb] != INF and distance[nb][dst] != INF:
                        new_cost = cost[src][nb] + distance[nb][dst]
                        if new_cost < distance[src][dst]:
                            distance[src][dst] = new_cost
                            next_hop[src][dst] = nb
                            updated = True

    # Step 4: Display the final routing table for every router
    print("\nFinal Routing Tables")
    for src in range(n):
        print(f"\nRouting Table for Router {names[src]}")
        print(f"{'Destination':<12}{'Cost':<8}{'Next Hop'}")
        for dst in range(n):
            hop = names[next_hop[src][dst]] if next_hop[src][dst] != -1 else '-'
            print(f"{names[dst]:<12}{distance[src][dst]:<8}{hop}")

    # Step 5: Let the user query the shortest path between any two routers
    src_name = input("\nEnter source router: ")
    dst_name = input("Enter destination router: ")
    src = names.index(src_name)
    dst = names.index(dst_name)

    if distance[src][dst] == INF:
        print("No path exists between the selected routers")
    else:
        print("Minimum Cost:", distance[src][dst])
        print("Shortest Path:")
        path = [names[src]]
        current = src
        while current != dst:
            current = next_hop[current][dst]
            if current == -1:
                print("Path cannot be determined")
                break
            path.append(names[current])
        else:
            print(" -> ".join(path))


main()
```

Output Screenshots

Run the program using the network from the worked example (A, B, C with costs A-B=2, B-C=3, A-C=8)

```
Initial Cost Matrix
A : ['0', '2', '8']
B : ['2', '0', '3']
C : ['8', '3', '0']

Final Routing Tables

Routing Table for Router A
Destination Cost    Next Hop
A           0       A
B           2       B
C           5       B

Routing Table for Router B
Destination Cost    Next Hop
A           2       A
B           0       B
C           3       C

Routing Table for Router C
Destination Cost    Next Hop
A           5       B
B           3       B
C           0       C
```

```
Enter source router: A
Enter destination router: C
Minimum Cost: 5
Shortest Path:
A -> B -> C
```

Students Observation

From this experiment, we learned how the Distance Vector Routing algorithm uses the Bellman-Ford method to compute the shortest, minimum-cost path between routers in a network. Each router starts out only knowing the cost to its directly connected neighbors, and by repeatedly exchanging and updating routing tables using the formula $Dx(y) = \min_v\{C(x,v) + Dv(y)\}$, the network eventually converges to the overall shortest paths. In the worked example, we observed that although router A had a direct link to C with cost 8, the algorithm correctly identified a cheaper path through B ($A \rightarrow B \rightarrow C$) with a total cost of 5, demonstrating that DVR does not simply rely on direct links but continuously improves its routing decisions using information learned from neighbors. This experiment gave practical insight into how distributed routing protocols like RIP, which are based on DVR, dynamically adapt routing tables as the network topology or costs change.

FAQ

1) What is the basic principle of Distance Vector Routing?

Each router shares its routing table (distance and next hop) with neighboring routers and selects the path with the lowest cost.

2) What is the difference between Distance Vector Routing and Link State Routing?

Distance Vector	Link State
Shares routing table with neighbors	Shares link-state information with all routers
Uses Bellman-Ford algorithm	Uses Dijkstra's algorithm
Slower convergence	Faster convergence
Simpler to implement	More complex

3) What is the Count-to-Infinity problem in Distance Vector Routing? Explain how it occurs and mention one method to prevent it.

It occurs when routers keep increasing the cost of a failed route because they incorrectly believe another router has a path to the destination.

Prevention: Split Horizon can be used to prevent this problem.

4) Which algorithm is used in Distance Vector Routing?

Bellman-Ford algorithm.

5) What happens when a network link fails?

The router detects the failure, updates its routing table, and informs neighboring routers. A new route is selected if available.